

RepoScout

A Field Guide to the Codebase

How every piece works, and why it was built that way

A reference iOS application demonstrating 2026 best practices

SwiftUI · Swift 6 concurrency (SE-0461) · Combine · SwiftData · Swift Testing · XCTest with BDD activities

Project: `testExample.xcodeproj` (product branded RepoScout, bundle ID `work.timmaher.RepoScout`)

Requires Xcode 26.2+ · iOS 26.0 · iPhone · English + German, tested in both · Zero third-party dependencies · MIT

Fourth edition – revised across successive independent audits; this document's source is committed at `docs/guide/`
August 2026

Contents

1. What RepoScout Is
2. Running the App and the Tests
3. The Map: Project Structure
4. The Life of a Search – an End-to-End Trace
5. The Model Layer: Three Shapes for One Repository
6. The Service Layer: a Protocol, Typed Errors, and Honest HTTP
7. LoadState: Stale-While-Revalidate and Failure That Keeps Your Work
8. SearchViewModel: One Funnel, One Dedup Key, One Scoped Ticker
9. The Views: Where Logic Was Deliberately Left Out
10. Persistence: SwiftData, @Query, and the Store Split
11. The Composition Root: Dependency Injection Without Magic
12. The Concurrency Model: Swift 6, SE-0461, and What "async" Actually Means Now
13. Combine's Role in 2026 – the Honest Boundary
14. Unit Testing with Swift Testing
15. UI Testing: BDD Scenarios, Two Languages, Audited Accessibility
16. Localization and Accessibility
17. Shipping Hygiene: the Parts That Aren't Code
18. Design Decisions, Trade-offs, and Known Edges

1 What RepoScout Is

RepoScout is a small, complete iOS app: you search GitHub's public repository API, browse results, open a detail screen, and save favorites that persist locally. That functionality is deliberately modest – the app exists to be *read*, not shipped. Every file demonstrates how a professional iOS team would build this in 2026, and the doc comments explain the *why* behind each pattern.

How to read this guide: Section 4 walks one keystroke through the entire system – read it first; every later section deepens one hop of that journey. Sections 5-11 are the design decisions worth stealing. Section 12 is the concurrency chapter – it explains SE-0461, the change to what `async` means that most material written before 2025 silently gets wrong. Sections 14-17 cover the testing and shipping practices that separate a demo from a product.

A revised edition. This codebase has been through repeated rounds of independent, adversarial audit (SwiftUI idiom currency, hostile concurrency review with compiler probes, professional-completeness review including a native-quality pass over the German localization, and a later round that went after the tests themselves – several of which turned out to be incapable of failing), each followed by a full remediation. The count is deliberately not given here: it would be wrong after the next one, and the record of every round is in `docs/process-archive/` anyway. Several sections teach lessons that exist *because* an audit caught an earlier version getting them wrong – those are told as "here is the mistake and why", because that is the most durable form of the lesson. This document's HTML source is committed at `docs/guide/`, so it can be corrected and re-rendered by anyone.

Naming note. The Xcode project, targets, and schemes are still named `testExample`; the product is RepoScout via display name, a real icon, and the bundle ID `work.timmaher.RepoScout`. Renaming a `pbxproj` is high-churn, low-value work; product identity and project naming are independent, and leaving them different demonstrates that.

2 Running the App and the Tests

Open `testExample/testExample.xcodeproj` in Xcode 26.2 or later and run the `testExample` scheme. The scheme is **shared** (committed at `testExample/testExample.xcodeproj/xcsreddata/xcschemes/testExample.xcscheme`), so these commands work on a fresh clone. All commands run from the `testExample/` directory that contains the `.xcodeproj`:

```
# Unit tests (Swift Testing) – runs under the test plan, in English AND German
xcodebuild test -project testExample.xcodeproj -scheme testExample \
  -destination 'platform=iOS Simulator,name=iPhone 17 Pro' \
  -only-testing:testExampleTests

# UI tests (XCUITest, BDD-style) – boots a simulator, slower
xcodebuild test -project testExample.xcodeproj -scheme testExample \
  -destination 'platform=iOS Simulator,name=iPhone 17 Pro' \
  -only-testing:testExampleUITests

# Everything, both languages (roughly 12 minutes)
xcodebuild test -project testExample.xcodeproj -scheme testExample \
  -destination 'platform=iOS Simulator,name=iPhone 17 Pro'
```

Substitute any installed simulator for the `name=` value. The scheme runs a committed **test plan** (`testExample.xctestplan`) with two configurations – English (`en / US`) and German (`de / DE`) – so every test invocation exercises both localizations, with code coverage collected for the app target – the full suite's figure sits in the mid-nineties, and the one-line command to read it off a result bundle is in the README rather than a number here that goes stale. Both locales are pinned explicitly: an empty options dictionary means "whatever the host machine is set to", so on a German-locale Mac both configurations would quietly run German – and the development-language-only tests would then be *reported as skipped*

in both configurations and executed in neither. That is the failure mode worth naming, because it does not look like one: the report is green, every skip carries its stated reason, and nothing anywhere says that the English pass those tests exist for never happened. The plan also enables per-test timeouts with a 120-second default allowance. The UI tests never touch the network: they launch the app with debug-only arguments (Section 11) that swap in a deterministic mock client and an in-memory database.

3 The Map: Project Structure

Path	Responsibility
testExample/testExample/RepoScoutApp.swift	@main entry point; owns AppDependencies
testExample/testExample/Models/	Repo (domain model, incl. the webURL validation seam) and the network DTOs + one mapping
testExample/testExample/Services/	GitHubClient protocol, GitHubClientError, LiveGitHubClient (@concurrent), MockGitHubClient (DEBUG-only actor)
testExample/testExample/Support/	LoadState, AppDependencies, PreviewSupport (a PreviewModifier), nonisolated shared Logger s
testExample/testExample/ViewModels/	SearchViewModel – the app's only view model
testExample/testExample/Persistence/	FavoriteRepo (@Model) and FavoritesStore (write-side repository, batch-aware)
testExample/testExample/Views/	RootView (TabView), Search/, Detail/, Favorites/
testExample/testExample/Localizable.xcstrings	String Catalog – English + full German, plural substitutions, translator comments
testExample/testExample/PrivacyInfo.xcprivacy	Privacy manifest (Section 17)
testExample/testExample.xctestplan	Two-language test plan; app-scoped code coverage
testExample/testExampleTests/, testExampleUITests/	Swift Testing suites + actor doubles; BDD UI tests + screen objects + accessibility audits
.github/workflows/ci.yml, .swift-format	CI workflow (written for a future remote; see §17) and the house-style record
README.md, ARCHITECTURE.md, LICENSE, docs/	Quick start, guided tour, MIT license, this guide's source, archived process reports

The project uses Xcode's *synchronized folder groups*: any file placed in a target's folder on disk automatically joins that target – nobody edits project.pbxproj to add files.

4 The Life of a Search – an End-to-End Trace

Suppose the user types "swift" into the search field.

```
keystroke → .searchable binding → SearchViewModel.searchText (didSet)
  → querySubject.send() → Combine: debounce(300ms, main) → filter(trimmed ≠
lastDispatchedQuery)
  → sink → dispatch(query) (records key, cancels old task, starts new – the ONLY entry point)
  → state = .loading (first load) or .loaded(stale, isRefreshing: true) (refinement)
  → await client.searchRepositories() (@concurrent – leaves the main actor here)
  → URLRequest (Accept, User-Agent, 15s timeout, + encoded as %2B) → JSON → RepoDTO → Repo
  → state = .loaded([Repo], isRefreshing: false) → @Observable invalidates SearchView
  → displayedRows? → ONE List of RepoRowView (same identity under .loaded and .failed-with-rows)
```

1. **The binding.** `.searchable` with autocapitalization and autocorrection disabled (repository names are identifiers – iOS would "fix" `vapor` into `vapour`), and the iOS 26 `.searchToolBarBehavior(.minimize)`. Return calls `submitImmediately()`, skipping the debounce the user has already outrun.
2. **Into the stream.** `searchText`'s `didSet` feeds a Combine `PassthroughSubject` – the moment a value becomes an *event in a stream*.
3. **The pipeline.** `debounce` waits out the typing; then a `filter` compares the *trimmed* candidate against `lastDispatchedQuery`. Why a filter and not `removeDuplicates()` is Section 8's lesson.
4. **One funnel.** The sink calls `dispatch(_)` – the only way any code starts a search. It records the dedup key, cancels the previous task, starts and stores the new one. Return, Retry, and pull-to-refresh all go through the same door, which is what makes "the newest search wins" a guarantee instead of a habit.
5. **State respects the screen.** A first load shows a spinner; a refinement keeps the stale results visible with a small indicator (`isRefreshing`); a refinement that *fails* keeps them too, with an error bar (Section 7).
6. **The network, off the main actor – explicitly.** The production client is `@concurrent`; under SE-0461 that attribute, not the `async` keyword, is what moves the request and decode off the caller's executor (Section 12).
7. **JSON becomes domain** via `Repo.init(dto:)` – the single seam where GitHub's vocabulary ends.
8. **The UI is a function of state – in two shapes, not one branch per case.** `SearchView.content` first asks whether there are rows worth reading; if there are, it builds a single `List` that both `.loaded` and `.failed(_, stale:, _)` share. Only the row-less states go through a `switch` (Section 9).

5 The Model Layer: Three Shapes for One Repository

Type	Shape belongs to	Changes when...
RepoDT0	GitHub's API	GitHub changes their JSON
Repo	This app's domain	The app's needs change
FavoriteRepo	The on-disk schema	A migration is written

Three types that rhyme, on three different clocks, connected by exactly one conversion each (`Repo.init(dto:)`, `FavoriteRepo.init(repo:)`, `FavoriteRepo.asRepo`). `Repo` is a `nonisolated struct`: `Identifiable` for `List`, `Hashable` for value navigation, `Sendable` so it can legally cross the executor boundary the `@concurrent` client creates. The DTOs use explicit `CodingKeys` over a global `snake_case` strategy – boilerplate traded for greppability.

Validation lives at the seam: `Repo.webURL` returns the repository URL only when its scheme is `http(s)` – compared case-insensitively, since URL schemes are case-insensitive per RFC 3986. The risk being guarded is a server-controlled custom scheme deep-linking into another installed app; the detail screen renders its GitHub link only through this property.

6 The Service Layer: a Protocol, Typed Errors, and Honest HTTP

```
nonisolated protocol GitHubClient: Sendable {
    func searchRepositories(matching query: String) async throws -> [Repo]
}
```

One method, `async`, returning domain models. Eight conformances exist across the project: the production client, the `DEBUG` mock actor, and six actor-based test doubles (Section 14) – and nothing downstream can tell them apart. That substitutability is the testing strategy.

The live client

`LiveGitHubClient` builds a real `URLRequest`: 15-second timeout, `Accept: application/vnd.github+json`, and a `User-Agent` (which GitHub requires). Two details the audits sharpened:

- **+ is percent-encoded by hand.** `URLComponents` legally leaves `+` raw in a query value, but GitHub form-decodes it as a space – so `c++` silently searched for `c`. The client rewrites `+` to `%2B` after building the components, and a test pins it.
- **Status codes are read the way GitHub actually sends them** (excerpt; comments abridged from the source):

```

case 200..<300: break // any 2xx is success
case 400, 422: throw .invalidQuery // GitHub's answer to a malformed/empty query
case 403: // overloaded: rate limit AND plain forbidden –
// claim "give it a minute" only when x-ratelimit-remaining / Retry-After say so
case 429: throw .rateLimited
default: throw .server(statusCode:)

```

Both transport catch clauses log through `Logger.network` – `debug` for the cancelled one, `error` for everything else – which is what makes the shared logger in `Support/Logging.swift` more than a declaration, and what makes the rogue- `CancellationError` story below diagnosable in the field rather than only in a test.

Cancellation is normalized – `URLSession` reports a cancelled task as `URLError(.cancelled)`, which the client surfaces as `CancellationError` so callers handle one shape. What the caller does with it matters more, and is Section 8's story: the view model's catch asks the cancellation *flag*, never the error *type*, because a `CancellationError` can arrive on a task that was never cancelled (session teardown), and trusting the type once produced an unrecoverable forever-spinner.

The mock client

`MockGitHubClient` is a DEBUG-only *actor* in the app target (previews and UI tests run the real binary). Scenarios, chosen by environment variable: `success`, `searchError`, `emptyResults`, `searchErrorOnce` – which fails the first completed call *per distinct query*, making the Retry-recovers UI test immune to typing speed (a slow keystroke burning the failure on `"s"` no longer steals it from `"swift"`) – and its mirror image `searchSucceedsThenFails`, whose first completed call per query succeeds and every later one fails. That last one exists to reach the only state no other scenario can: results on screen *under* a failure banner, and the retry taken from there. Its 300ms delay sits before the scenario switch, so a cancelled mock call throws like the real client would.

7 LoadState: Stale-While-Revalidate and Failure That Keeps Your Work

```

nonisolated enum LoadState<Value: Sendable & Equatable>: Equatable, Sendable {
    case idle
    case loading
    case loaded(Value, isRefreshing: Bool)
    case failed(message: String, stale: Value?, isRefreshing: Bool)
}

```

The core argument is the classic one: a bag of `isLoading` / `items` / `error` properties has eight combinations, most nonsense; a closed enum has exactly one state and the compiler guarantees every case a screen. What the audit rounds added is that the *naïve* four-case version of this enum fails its own sales pitch three times:

- **isRefreshing on loaded** – without it, refining a query must either blank the results to a spinner (the screen flickers on every debounced keystroke – the v1 behavior) or lie about the request being done. "Results on screen while a refinement is in flight" is a legal, necessary state. The Bool lives *inside* `loaded` so "refreshing with nothing to refresh" stays unrepresentable.
- **stale on failed** – a refinement failure is not a reason to throw away the last thing that worked. With results showing, failure renders as a compact error bar *over the still-readable results*; a first-load failure carries `nil` and selects the full-screen error. Optional rather than a fifth case, so the view still has one failure branch.
- **isRefreshing on failed too** – a failure can be revalidating, and pretending otherwise cost real correctness (below). While the banner's own Retry is in flight the screen is still a failure: banner up, rows kept, spinner on. That is one state, and now it is one value.

Which failures get to keep their rows, and which searches get them back. `stale` answers the first question; a gate in `SearchViewModel.search(matching:)` answers the second, and it took three rounds to get right. When a search begins from a `.failed` that kept non-empty rows, those rows *stay* – the state becomes `.failed(sameMessage, stale: rows, isRefreshing: true)` – but only when the incoming query is either the one the rows *answer* (`lastCompletedQuery` : what pull-to-refresh re-runs, and what re-submitting the same text produces) or the one the banner is *about* (`lastFailedQuery` : what the banner's own Retry re-runs). Everything else starts blank.

Both halves are scar tissue. Ungated, a brand-new query typed from a failure screen resurrected the previous search's rows and presented them as its own, with a spinner claiming they were being refreshed – and under rate limiting the same ancient rows were carried forward again on every failure. Gated on `lastCompletedQuery` alone, a failed *refinement* blanked its rows the instant the user tapped the Retry the banner was offering, which is the founding case of the entire feature. Two keys, two witnesses in the unit suite: delete either arm of the condition and exactly one test fails.

Why the retry stays failed – the third payload's whole reason. The gate above is *asymmetric*: the `failed` branch asks whether the incoming query has a claim on the rows; the `loaded` branch asks nothing, because rows in a genuine `.loaded` always belong to the query being refreshed. Earlier code promoted a retrying failure to `.loaded(stale, isRefreshing: true)` for the duration of the request – and that broke the premise the ungated branch rests on. A query dispatched *during* the retry (a keystroke, a pull) read a `.loaded` that had never loaded anything, took the ungated branch, and adopted rows answering a different question as its own; its failure then carried them forward as *its* stale results, and the next retry laundered them again. Terminal states cannot see it – the laundering happens mid-flight – so the witness is a unit test that holds the client open: `keptRowsCannotOutliveTheRetryTheySatUnder` asserts the interloping query goes to `.loading` and fails with `stale: nil`. Restore the promotion and it fails on both.

Two companion details: the empty-results title comes from `lastCompletedQuery` – the query that *produced* the empty set – never from the live field text, which is still ahead of the debounce and may describe a search nobody has run; and the empty branch renders `isRefreshing` as a small overlay spinner instead of discarding it with a wildcard, so refining from an empty set visibly does something.

8 SearchViewModel: One Funnel, One Dedup Key, One Scoped Ticker

dispatch(_:) – the only door

```
@discardableResult
func dispatch(_ query: String) -> Task<Void, Never> {
    lastDispatchedQuery = query.trimmingCharacters(in: .whitespacesAndNewlines)
    searchTask?.cancel()
    let task = Task { await self.search(matching: query) }
    searchTask = task
    return task
}
```

`search(matching:)` is **private**; the debounce sink, `submitImmediately()`, `retry()`, and pull-to-refresh all go through `dispatch(_)`. This came out of an audit finding, and the reasoning is worth keeping: the design's safety rests on two invariants – *every in-flight search is `searchTask`* (so cancelling it is a complete statement), and *the dedup key always describes the search actually running*. When `search` was internal and did its own bookkeeping-free work, those held only by convention; the first future caller (pull-to-refresh was the obvious one) would have broken them into a genuine stale-clobbers-fresh bug. Funnelling makes the invariants structural, and returning the task lets tests and `.refreshable` await completion without touching internals.

Why `filter`, not `removeDuplicates()`

`removeDuplicates()` dedups on the *input*, which is only sound when the operation is deterministic – and a network search is not. With it, a failed search became unrepeatable by typing: the operator remembered the failed query forever. The replacement dedups against `lastDispatchedQuery`, which the *outcome* controls: recorded on dispatch (so a Return-then-debounce pair can't double-fire), cleared on failure (so retyping retries), kept on success (so identical re-submissions stay suppressed), and trimmed on both sides (so "swift " isn't a "different" query worth a second request).

One catch, asking the flag

The failure path has a single `catch`, and it checks `Task.isCancelled` – never `error is CancellationError`. The distinction is load-bearing: a `CancellationError` can arrive on an *uncancelled* task (the client normalizes every `URLSessionError(.cancelled)`, which `URLSession` also produces for session teardown). An earlier version trusted the type and could exit leaving `.loading` on screen forever – with the dedup key still pinned, so retyping was filtered, and no Retry button, because Retry renders on `.failed`. Asking the flag covers both worlds: a genuinely superseded search returns silently; anything else becomes a `.failed` the user can act on. The post-await `guard !Task.isCancelled` before the success writes is sound for a reason worth stating precisely, because the reassuring version of it is false: it is *not* that only main-actor code can cancel us. `Task.cancel()` is `nonisolated` – anybody on any executor may call it, and this suite's own tests do. What holds is **turn atomicity**: the guard and the three writes after it are one uninterrupted main-actor turn with no suspension point between them, so a cancellation observed

afterwards cannot un-write them, and the only canceller in production is a newer `dispatch(_:)` that has already replaced `searchTask` and will write its own result over ours. The worst case is a state that is briefly this search's and then immediately the newer search's – never a torn mixture. The comment in the source says exactly that, having previously said the comfortable thing instead.

The `catch` also records `lastFailedQuery`, in the same turn as the `.failed` it writes. That is what lets the promotion in Section 7 distinguish "the query this banner is about" from "the query these rows answer" without re-reading state after an await.

The ticker

Combine's second appearance: `Timer.publish`, started/stopped by the results list's appear/disappear (an init-started timer on an app-lifetime object would fire forever on every tab), seeded on start (`Timer.publish`'s first emission is a second away; without the seed, returning to the tab showed a stale timestamp for a beat), and read only by a leaf view (Section 9). The `max(0,...)` clamp's comment is now honest: it *does* fire – the first body evaluation runs before `.onAppear` seeds the clock – and in steady state `Int` truncation is what absorbs sub-second lag, not the seed.

Type-checks vs. is-safe. The Combine sink closures are marked `@MainActor`, and the comments name scheduler: `DispatchQueue.main` / on: `.main` as load-bearing. Precision matters here: default `MainActor` isolation is why the closures *type-check* touching state; the scheduler arguments are why that's *true*; and the `@MainActor` annotations insert a *dynamic* check – nothing is verified at compile time, and that is the point. Swap the scheduler for a background queue and the code compiles clean, then traps loudly at runtime instead of quietly corrupting main-actor state. An audit verified the trap empirically.

9 The Views: Where Logic Was Deliberately Left Out

SearchView

Two shapes, not six branches – and the difference is structural, not stylistic. The view first asks `displayedRows` whether there are rows worth reading: a non-empty `.loaded`, or a `.failed` that kept non-empty stale rows. If there are, the screen is **one** `List`, and those two states are the *same view identity*. Only the row-less states go through a four-arm `switch`.

State	UI
<code>.loaded(repos, isRefreshing)</code> , non-empty <code>.failed(_, stale, isRefreshing)</code> , non-empty stale kept	One List for both. It carries <code>.refreshable</code> (safe because the pull goes through <code>dispatch</code> , and it re-runs <code>lastCompletedQuery</code> rather than the live field), the ticker's lifetime, the VoiceOver announcement, and one <code>safeAreaBar</code> . Under a failure the bar is the <code>ErrorBanner</code> above the timestamp, stacked – how old these rows are matters most exactly when they are stale. The spinner beside that timestamp reads <i>either</i> case's refresh flag, which is how the banner's own Retry reports itself without the banner having to vanish.
<code>.idle</code>	<code>ContentUnavailableView</code> invitation
<code>.loading</code>	Full-screen spinner – only a load with nothing to keep blanks the screen
<code>.loaded</code> , empty	System no-results view titled with <code>lastCompletedQuery</code> , or the untitled variant when there is none; overlay spinner while refreshing
<code>.failed</code> , nothing to keep	Full-screen error with a prominent Retry

Why one list rather than two identical ones in two arms: a `List` built in one `switch` arm and a textually identical `List` built in another are two *different* views to SwiftUI, so every crossing destroyed one and built the other. Scroll position reset, rows cut instead of animating, and every modifier attached below restarted – which is how the failure state used to have no `.refreshable` and no timestamp footer at all, and how the ticker stopped and restarted on each transition. `FavoritesView` teaches the same lesson from the other side: one `List` for its whole lifetime, with the empty state as an overlay.

Be exact about what the compiler checks: the `where` clauses in `displayedRows` are **not** verified. A `switch` whose arms are all guarded is not exhaustive – it compiles because of the unguarded `default` – so nothing in the type system confirms those two guards describe the states they claim to. The tests do. And the screen is not a pure function of the enum alone: two readers reach for state that lives beside it (`lastCompletedQuery` in the empty branch and in `.refreshable`, `lastRefreshed / now` in the footer), which is the price of keeping `LoadState` a generic lifecycle enum that knows nothing about searching.

One accessibility consequence of the single identity is worth naming, because it is invisible: a failure that keeps its rows changes nothing the VoiceOver cursor is on, so nothing is spoken. `SearchView` therefore posts an `AccessibilityNotification.Announcement` when such a failure *arrives*, and the discriminator is the previous state's refresh flag rather than its case. Arriving from `.loaded` is a new failure; arriving from a failure that was already refreshing is a retry landing, with a message that may well have changed. What stays silent is the retry *starting* – `.failed(m, rows, false) → .failed(m, rows, true)` – where nothing audible has changed. (Testing "was the previous state `.failed`?" was right only while the retry laundered itself through `.loaded` on the way; it does not any more, and the old test would now swallow the re-failure it was written to announce.) The full-screen failure branch needs no announcement for the opposite reason: replacing the content relocates focus, which the system speaks by itself.

The banner earns two pieces of layout in its own right. Its Retry carries `.frame(minHeight: 44)`: a `.bordered` button sized for footnote text measures around 34pt, under the audit's 44pt minimum touch

target, and the banner being small is no reason for the way out of it to be. And the whole banner is a `ViewThatFits(in: .horizontal)` over a shared `@ViewBuilder` fragment – the same reflow as `RepoRowView.stats`, in the place it bites hardest. Side by side, a sentence-long message and a button share one line; at accessibility text sizes the message takes the space and the button squeezes toward its title's intrinsic width, so "Retry" truncates to "Ret..." – a control whose label no longer says what it does. The horizontal candidate is tried first and the stacked one takes over when it stops fitting, and `testTheFailureBannerSurvivesTheLargestDynamicTypeSize()` audits this screen at AX5 with the `.textClipped` suppression removed, which is what makes suppressing it at default size honest. The banner also arrives and leaves on a `.move(edge: .bottom).combined(with: .opacity)` transition, animated on the *presence* of a failure rather than on `state` – keying on the enum would re-run the animation for every message change and for both edges of the retry's refresh flip, animating a bar that is not going anywhere.

The footer and error bar use `safeAreaBar` – the iOS 26 API that supplies the system bar background and scroll-edge treatment – rather than `safeAreaInset` with a hand-painted `.background(.bar)`, which on iOS 26 rendered a flat slab above the floating tab bar. Both error presentations carry the same two accessibility identifiers (`search.errorView` on the message, `search.retryButton` on the button); only one renders at a time.

A real-world scar, kept on purpose. The error identifier lives on the description text, never the container:

`ContentUnavailableView` re-parents a container-level identifier onto its merged children, which clobbered the Retry button's identifier at runtime. The interim workaround – matching the literal English "Something went wrong" – then broke the moment the app grew German. Identifier on the one true leaf: correct and locale-independent.

LastRefreshedFooter – invalidation confined to a leaf

`@Observable` attributes property reads to whichever `body` performs them. The footer text reads `now`, which the ticker rewrites every second – inlined into `SearchView` (even inside the bar's builder closure) that read would belong to `SearchView.body`, re-evaluating the whole screen once a second. As its own small struct, the per-second dependency belongs to a view whose body is a spinner and a `Text`. The comment is precise about two things the first edition fudged: *re-evaluation* is not *re-rendering* (structural diffing elides unchanged rows – the cost is real but smaller than "the whole screen re-renders"), and the insulation is one-directional (the leaf still re-runs when the parent invalidates; that direction is harmless). The spinner is no longer gated on the timestamp's existence – progress and "a previous request finished" are unrelated facts – and the whole bar is `.accessibilityHidden(true)`: a label that changes every second is hostile to VoiceOver, and this is ambient status, not content.

RepoRowView

- Star counts render via `.formatted()` (locale grouping) – the v1 raw interpolation both skipped grouping and minted a meaningless bare `%lld` key into the catalog.
- `ViewThatFits` reflows the stats row vertically at accessibility text sizes; the summary's line limit lifts when `dynamicTypeSize.isAccessibilitySize`.

- De-emphasized text uses a semantic **asset-catalog color** with Any/Dark *and* Increase Contrast variants – replacing a fixed `opacity(0.65)` that passed the contrast audit but opted out of Increase Contrast and was only valid against the backgrounds it was measured on. (Stock `.secondary` at caption sizes measures ~3.9:1, below WCAG AA's 4.5:1 – the automated audit caught it.)
- One VoiceOver element per row (`children: .ignore` – the label fully replaces the fragments), speaking whole localized sentences with correctly *inflected* star counts via catalog plural substitutions (Section 16).

RepoDetailView

No view model – no async orchestration to justify one. Live favorite state via a parameterized `@Query` built in `init`; writes through `FavoritesStore`. The toolbar button carries a *stable* label ("Favorite") with state in the `.isSelected` trait – the earlier flipping label plus trait read as the contradiction "Remove from Favorites, selected". The star swap animates via `.animation(value: isFavorite)` bound to the *observed* value (a `withAnimation` around the write animated nothing: the value changes later, when SwiftData's notification lands), and the haptic is driven by *recorded intent* rather than by the observed value. The direction is captured at tap time (`lastToggleAddedFavorite`) and the trigger counter advances only after the store write succeeds, so the buzz describes what the user just did and a failed toggle produces no confirmation at all. Reading `isFavorite` to choose between `.success` and `.impact` was the bug: that is a `@Query` result, which lands on a *later* update than the tap – as the `.animation(value:)` on the very same button already documented – so the feedback described the state before the write, and merely *navigating* to a favorited repo could buzz. The GitHub link renders only through `Repo.webURL`.

FavoritesView

Live `@Query` reads; the empty state is a non-hit-testable `.overlay` so the List keeps its identity and the last row's delete animates; `EditButton` appears only when there is something to edit; multi-row deletes go through the store's batch API – one `context.save()`, one change notification, one list rebuild.

The screen **owns its edit mode**: `@State private var editMode`, injected with `.environment(\.editMode, $editMode)` placed outside `.toolbar`. That is not decoration, it is the fix for a real trap. Deleting the last favorite hides the `EditButton` – the toolbar only shows it when there is something to edit – and hiding the only control that can leave edit mode does not leave it, so the next row arrived into a list still editing, with the restored button reading "Done". Writing `@Environment(\.editMode)` and resetting it does not work: the binding accepts the write, reads back exactly what was written, and the toolbar's `EditButton` goes on saying "Done" anyway – measured from the declaring view and from a child inside the `NavigationStack`, with an on-screen probe, before the shape changed. One source of truth that the button, the rows and the reset all share is the version that works, and a UI test picked the winner.

RootView and previews

The search tab declares `role: .search`, with a comment that now says precisely what that buys and what it doesn't. The two behaviours that usually accompany it are separate opt-ins in two different files:

`.tabBarMinimizeBehavior(.onScrollDown)` sits on the `TabView` here in `RootView`, while the minimize-into-search behavior is `.searchToolBarBehavior(.minimize)` in `SearchView`, on the same view as the `.searchable` field it configures. Both tabs carry accessibility identifiers via `TabContent.accessibilityIdentifier` – an API two earlier comments confidently claimed didn't exist (it has since iOS 18; the audit that caught this is why the tab query in UI tests is now locale-independent). Previews are seeded directly into every interesting state through a DEBUG-only seam (a preview renders once – it cannot wait out an async search, which is why the old "Error" preview silently showed the idle prompt forever), and the SwiftData container arrives via a `PreviewModifier` trait (`traits: .sampleData`), the iOS 18+ replacement for a global container constant.

10 Persistence: SwiftData, @Query, and the Store Split

Reads via `@Query` in views – live, animated, no refresh bugs. Writes via `FavoritesStore` – dedup, save timing, and batching in one tested place. Sharpened details:

- **.unique is a safety net, not the mechanism.** The dedup the app relies on is the fetch-then-branch in `existingFavorite(for:)`. Remove the attribute and every test of the app's *behavior* still passes – one test exists precisely to pin the constraint itself (a direct duplicate insert collapses to one row).
- **isFavorite throws** – its earlier `try?` conflated "the fetch failed" with "not favorited".
- **remove(_ favorites:)** deletes N rows under a single save.

Unit tests run against a fresh in-memory `ModelContainer` per test; UI tests get the same hermeticity via a launch argument.

11 The Composition Root: Dependency Injection Without Magic

`AppDependencies` decides every concrete implementation once, at launch. Both test hooks (– `UITestMockNetwork`, `-UITestInMemoryStore`) are `#if DEBUG`-gated – an audit caught the in-memory flag, a data-loss switch, being honored in Release. And the container build does not `fatalError` on the failure that actually happens: the most common real-world container failure is a schema-migration mismatch after an app update, where crash-on-launch is an inescapable crash loop. The persistent path therefore degrades – the root logs at fault level and falls back to an in-memory store, alive but with an empty favorites list. There is one `fatalError` left, and it is deliberate: if the *in-memory* container also fails to build, the schema itself is malformed. That needs no disk, no migration and no permissions, so it fails identically on every launch on every device – a programmer error present in the binary, not a condition a running app can degrade around. Constructor injection for the view model; `.modelContainer` for SwiftData; the release binary contains none of the test machinery.

12 The Concurrency Model: Swift 6, SE-0461, and What "async" Actually Means Now

The project compiles in Swift 6 language mode with `SWIFT_DEFAULT_ACTOR_ISOLATION = MainActor` and `SWIFT_APPROACHABLE_CONCURRENCY = YES` – Xcode 26's defaults. The second setting enables **SE-0461**:

The rule that changed. A plain `nonisolated async` function no longer hops to a background executor – it runs on its *caller's* executor. If a `MainActor` view model awaits your `nonisolated async` networking function, its body – including your JSON decoding – executes **on the main actor**. It compiles, passes tests, and drops frames. The first edition of this codebase had exactly that bug, wearing the pre-2025 folklore ("decoded on a background `URLSession` task") as a doc comment. The fix is one attribute: `@concurrent`, which opts the function back out to the global concurrent executor. Both facts – the silent main-actor execution without the attribute, and the genuine hop with it – were verified with compiler probes during the audits, including through an `any GitHubClient` existential.

Where the annotations live: UI-facing code needs none (`MainActor` by default – which is also why `FavoritesStore`'s fetch-then-branch has no race: two rapid taps are two serialized main-actor turns). `nonisolated` marks the crossing types – models, DTOs, the client protocol – and, less obviously, the shared `Logger` constants: a `static let` in an extension picks up the module's default isolation, and an accidentally-`MainActor` logger is unusable from exactly the `@concurrent` context where network failures live. Actors hold genuinely shared mutable state: the mock client and the test doubles.

For readers returning from the pre-SwiftUI era, the translation in brief – each row is one of this codebase's demonstrations:

Pre-SwiftUI technique (c. 2019)	RepoScout does it with...
UIViewController + delegates	SwiftUI View structs; state drives UI
KVO / NotificationCenter	<code>@Observable</code> per-body read tracking (§9's leaf-view lesson)
GCD, <code>DispatchQueue.main.async</code>	<code>async/await</code> + actors + <code>MainActor</code> default, with <code>@concurrent</code> as the explicit "off-main" verb
Core Data + NSFetchedResultsController	SwiftData <code>ModelContainer</code> + <code>@Query</code>
NSTimer you must invalidate	<code>Timer.publish</code> with an owned <code>AnyCancellable</code> , started/stopped with the UI
OCMock-style mocking	Protocol conformances + actor doubles; no framework

13 Combine's Role in 2026 – the Honest Boundary

Combine appears exactly twice – the debounced search pipeline and the scoped ticker – both genuine *streams of events over time*. Every one-shot request/response is `async/await`; `LiveGitHubClient` contains

no Combine and says so. The boundary rule: *if a value arrives once, `await` it; if values keep arriving, model them as a stream.*

Two honest footnotes. First, operators encode assumptions: `removeDuplicates()` looked like exactly the right operator for the pipeline and was quietly wrong for a non-deterministic operation – the lesson is to check the assumption, not just reach for the obvious operator. Second, `AsyncSequence` / `AsyncStream` (plus `swift-async-algorithms` 'debounce') are the concurrency-native alternative a 2026 team would also evaluate; this app teaches Combine deliberately – it ships in the OS, the operators needed are built in, and it remains what this app's readers will inherit in existing codebases. A curriculum choice, stated as one.

14 Unit Testing with Swift Testing

Swift Testing throughout – `@Suite`, `@Test`, `#expect` / `#require`, parameterized cases – with `@MainActor` annotations where suites touch main-actor types. The supporting cast in `TestSupport.swift` is the real teaching content, and its roster is worth stating precisely:

- `SpyGitHubClient` – records queries, returns a canned result.
- `ScriptedGitHubClient` – a per-call result script; the mechanism behind the Retry-recovers test.
- `GatedGitHubClient` / `KeyedGatedGitHubClient` – suspend requests on continuations until the test says `open()`, globally or per-query; both are cancellation-safe via `withTaskCancellationHandler` (including the pre-opened path).
- `UncancellableGatedGitHubClient` – **deliberately ignores cancellation**. It exists because a cancellation-safe gate can never produce the scenario the post-await guard exists for: a *successful* response arriving on an already-cancelled task. The only double that can prove the stale result never lands is one that refuses to be cancelled.
- `RogueCancellationGitHubClient` – throws `CancellationError` on a task that was *never cancelled*: the forever-spinner regression's exact shape (Section 8), and the only double that can prove the catch asks the flag rather than the type. Together with its uncancellable sibling, the pair is the codebase's best test-design lesson: each exists to produce precisely the one input that distinguishes correct cancellation handling from plausible cancellation handling.

Infrastructure honed by the audits: `poll(until:)` re-checks its condition once after the final sleep and *throws* on timeout (the earlier version recorded an Issue and carried on – a timed-out wait could fall through into further awaits and hang the test instead of failing it); `StubURLProtocol` 's handler sits behind a real `Mutex`, and `withStubbedClient` additionally serializes whole invocations process-wide – the `Mutex` removes the data race with `URLSession`'s loader thread, the gate is what prevents two concurrent suites from answering each other's requests. A caller that gives up waiting for that gate now *throws* rather than proceeding: installing a handler you do not own corrupts the test that does, which is a worse outcome than the timeout it was reacting to.

Coverage worth naming: the full HTTP mapping table (403 with and without rate-limit headers, 422, 2xx incl. 203, garbage-200, `URLError(.cancelled)`, a clean round-trip); `+` encoding; the supersession

invariant driven through the *real* `dispatch` path; failure-then-retype re-fires while success-then-retype doesn't; an uncancelled `CancellationError` lands in `.failed` (the forever-spinner regression test); stale results surviving a failed refinement, and both arms of the gate that decides which searches get them back (re-running the query the rows answer, re-running the query the banner is about, and a genuinely new query still starting blank); the query dispatched *into* an in-flight banner retry, which must inherit nothing (§7); the empty-refinement state; ticker seeding, double-start, and stop-when-never-started; duplicate inserts; batch remove; and the localized plural labels, executed in both languages by the test plan.

15 UI Testing: BDD Scenarios, Two Languages, Audited Accessibility

The shape is unchanged: `Given/When/Then` wrapping `XCTContext.runActivity` so reports read as scenarios; screen objects encapsulating queries; hermetic DEBUG-only launches. What the rounds added:

- **Scenarios that act, not just look** – Retry is *tapped* and recovery asserted (via the per-query fail-once mock); empty results has a scenario; deletion is exercised through the visible `EditButton` path.
- **Keyboard-focus waits** before `typeText` – the classic CI flake, addressed.
- **Automated accessibility audits** – `performAccessibilityAudit()` across the screens, including the failure-banner-over-results screen, which is the only one that puts a *control* in the safe-area bar. Two of the four audits run at AX5 with the `.textClipped` suppression dropped, one for the results list and one for the banner over it, and those are the evidence for suppressing it at default size – a suppression covering two reflowing layouts needs a witness per layout, not one witness and a paragraph. The "Clear text" hit-region suppression still applies at AX5, because `UISearchTextField` builds that button and `.searchable` offers no hook to resize it at any text size. The audits caught a real WCAG contrast failure (§9) and an undersized Retry target (§9).
- **Two languages, honestly.** The test plan runs the suite in English and German. Queries the app controls are identifier-based and locale-independent – including the tab bar, once the false "no API for that" claim was corrected. The German configuration runs the entire suite *minus eight explicit skips*, and phrasing it that way is deliberate: an absolute test count in a document goes stale the first time somebody adds a test, while the number of things that cannot run in German is a fact about the app. Six of the eight match strings *Apple* owns (the system no-results view, `EditButton`'s title, the delete confirmation, and the search field's clear button, which all four audit tests share) and use the `XCTSkip` gate that says so. The other two are the screenshot gallery below, which is produced once, in the development language, and gates itself with its own sentence rather than borrowing a reason that would be false for it. Every one is visible in the test report as a skip, with its reason, rather than silently green.
- The launch test records an `XCTApplicationLaunchMetric` – a baseline-able number in the result bundle (a metric without a stored baseline records rather than asserts; the honest description) – plus the appearance-matrix screenshots.
- **The README's screenshots are test output.** `ScreenshotGalleryUITests` walks the app to four screens – results, detail, populated favorites, and results again under German – and attaches an

`XCUIScreenshot` at each, with `.keepAlways` lifetime so a *passing* run keeps them. `xcrun xcresulttool export attachments` lifts them out of the result bundle. A screenshot taken by hand starts lying the first time a view changes; one taken by the suite, from the same hermetic launch and the same fixtures as every other test, is regenerated by re-running one command. These are the only tests here that assert nothing beyond "the app reached this screen", and the type comment says so.

16 Localization and Accessibility

A catalog that localizes

The String Catalog carries English and a **complete German localization** – reviewed at native quality during the audits (the review found one grammar error and a dozen register improvements, all applied: consistent *du*-form, Apple-German terminology like *Inhaber* and *Internetverbindung*, Duden plurals – and a later pass applied four more: aligning *Suchanfrage* with its neighbouring string, pluralizing an enumeration German pluralizes, shortening the one sentence that blamed the app for something the user cannot act on, and rewording the Favorites empty state from "add to your favorites" to "mark as a favorite" so it names the affordance the way the affordance names itself). Every key has a translator comment; the app's name is marked `shouldTranslate: false` rather than "translated" to itself.

Plurals are done properly, at two levels. The footer string is spelled out ("Updated 42 *seconds* ago") with `one / other` variations in both languages – an inflecting language needs a noun to inflect. And the VoiceOver row sentences use catalog **substitutions** (`%#@starCount@`): the raw integer drives plural selection so German hears "1 Stern" / "67000 Sterne" – the technique an earlier doc claimed impossible, now demonstrated. The stated trade: the substitution's `%lld` drops digit grouping in the *spoken* count; the visible label still uses `.formatted()`.

Accessibility as architecture

- Rows are single VoiceOver elements speaking whole localized sentences – fragment-joining bakes in English word order and hands translators pieces they can't reorder.
- Dynamic Type genuinely accommodated: `ViewThatFits` reflow in both places a row of text and a fixed-width neighbour compete for one line – the row's stats and the failure banner's message-plus-Retry – and accessibility-size line-limit lifts. Both are audited at AX5 with the clipping check switched back on.
- Contrast measured, then fixed *semantically* – an asset color with Increase Contrast variants, not a hardcoded alpha.
- State conveyed as state: stable "Favorite" label + `.isSelected`, haptics driven by the user's action.
- Ambient noise silenced: the once-per-second footer is invisible to VoiceOver.
- And a correction the docs now carry: `accessibilityIdentifier` is a test-automation hook, invisible to assistive technology – identifiers are the UI tests' contract, not an accessibility feature.

17 Shipping Hygiene: the Parts That Aren't Code

Artifact	Why it's there
Shared scheme + <code>testExample.xctestplan</code>	Auto-created schemes are per-user: without a shared one, every documented command fails on a fresh clone. The test plan adds the English/German configurations – both locales pinned explicitly, so a non-English host cannot silently collapse them into one – scopes code coverage to the app target (mid-nineties for the full suite; read it off a result bundle rather than trusting this sentence), and enables per-test timeouts with a 120-second allowance.
<code>LICENSE</code> (MIT)	No license means all-rights-reserved – readers would be legally barred from copying the patterns the app teaches.
<code>PrivacyInfo.xcprivacy</code>	Required when an app uses required-reason APIs or listed SDKs; this app uses neither, and declaring the all-empty manifest – "we collect nothing," stated rather than implied – is the instructive case.
Real app icon + accent color	1024px light/dark/tinted; dark and tinted ship as artwork on transparency so the system composites its own backgrounds, per Apple's guidance (an earlier export baked opaque plates in). The three-PNG appearance set is a deliberate choice over an Icon Composer <code>.icon</code> bundle: the <code>.icon</code> format is the richer one – it carries layers the system can re-composite for glass, tint and dark modes – but it is an opaque binary produced by a GUI tool, and a codebase whose point is that every artifact can be read and reasoned about in a diff is the wrong place for one. The accent color is an asset with light, dark, and both Increase Contrast variants, for the same reason <code>Deemphasized</code> is (§9).
<code>ITSApUsesNonExemptEncryption</code> <code>= NO</code>	Removes a manual export-compliance question from every upload.
App category (Developer Tools)	Optional in the Info.plist for iOS (the store listing sets it), declared here so archives carry it.
ATS fully strict	No exception dictionary – <code>api.github.com</code> passes default ATS (verified with <code>nsurl</code>). Sample apps that blanket-disable ATS teach the wrong reflex.
iPhone-only, no committed team ID	Device family matches what was designed; a personal team ID neither leaks nor breaks other people's signing.
<code>IPHONEOS_DEPLOYMENT_TARGET</code> = <code>26.0</code>	Lowered from the 26.2 the template wrote. Availability is checked against this number on every build, so lowering it and building clean is the evidence that nothing here needs the higher one – which was excluding devices in exchange for nothing.
<code>.github/workflows/ci.yml</code>	A unit + UI pipeline plus an advisory formatter job, written for when the repo gains a remote – it has not yet executed, and its header says so. The UI job writes a result bundle and uploads it with <code>if: always()</code> , because a UI failure on a hosted runner is otherwise a log line with the screenshots left on a machine about to be destroyed – tarred first, because <code>upload-artifact</code>

uploads the files *inside* a directory and an `.xcresult` is a directory that only `xcresulttool` can open with its wrapper intact. The unit job writes one too and prints coverage by target, so the figure this guide declines to quote is in every run's log. All three jobs pick their toolchain with a job-level `DEVELOPER_DIR` rather than three copies of `sudo xcode-select`, and each opens by printing the Xcode versions it can see – the first thing to break on a runner image update is that path, and its failure otherwise says nothing about its cause.

`.swift-format`

The house style as a checkable record – and an honest one. `xcrun swift-format lint --recursive testExample`, run from the repository root and identical to the CI line, reports 265 diagnostics over twelve files: 177 `Indentation`, 77 `AddLines`, 9 `LineLength`, 2 `Spacing`. The first two – 254 of the 265 – are the pretty-printer's opinions about wrapping call arguments and collection literals, which this codebase declines, and they grow when a long line is hand-wrapped, so the total is not a quality score. All nine long lines are `String(localized:comment:)` translator comments, whose `StaticString` argument cannot be split without putting a newline in the comment a translator reads or telling them less; every long line that could be broken has been, so nine is the count and the whole explanation at once. Advisory in CI (`|| true`), never a gate – and no `--strict`, since its only effect is an exit status `|| true` throws away.

`docs/process-archive/`,
`docs/guide/`,
`docs/screenshots/`

Point-in-time build/remediation reports, disclaimed; this guide's editable source; and the README's gallery, regenerated from the UI suite rather than taken by hand.

18 Design Decisions, Trade-offs, and Known Edges

Decisions worth restating

- **A view model only where it earns its keep** – one screen has orchestration; one screen has a view model.
- **State as a closed enum including the states everyone forgets** – stale-while-revalidate *and* failure-that-keeps-your-work.
- **One funnel for every search** – invariants made structural, not habitual.
- **Ask the flag, not the type** – cancellation handling that survives errors lying about their origin.
- **Rows may be kept, but only under a query with a claim on them** – the gate is two keys wide (the query the rows answer, the query the banner is about) because one key was too narrow and none was too wide. Middle grounds in a controller are usually a smell; this one is the shape of the problem.
- **A state must not lie about itself, even briefly** – a retrying failure stays `failed` for the whole retry rather than borrowing `loaded`'s shape for the duration. The gate above trusts `loaded` precisely because `loaded` is supposed to mean something; a transient lie in one branch became a wrong screen in another (§7).

- **A test that cannot fail is not a test** – a whole round was spent proving, by running the mutant, that each rewritten test fails when the code it claims to protect is deleted.
- **The protocol seam is the testing strategy** – eight conformances, no mocking framework.
- **Comments must be true under the actual build settings** – both audits' deepest findings were confident prose on correct code; in a teaching codebase a wrong comment is the defect readers copy.

Known edges (deliberate, documented)

- No pagination, auth, image loading, or iPad layout – scope, by design, with the device family matching it.
- Eight UI test cases skip outside English, for two different reasons: six match Apple-owned strings with no identifier hooks, and two produce the README's screenshot gallery, which is taken once in the development language. Both skips are explicit, and each says which reason it is.
- The edit-mode delete control is queried by its system image name – `List` publishes it beside the row, outside any app-controlled element; measured, and documented in the screen object.
- The spoken star count loses digit grouping to the plural substitution; the visible count keeps it.
- CI has never executed (no remote yet); `swift-format`'s pretty-printer wrapping is advisory, and its CI job is explicitly `|| true`; the searchable field's own identifier limits are noted in the screen object and unverified against newer SDKs.
- The banner's arrival animation is asserted only indirectly: the audit and scenario tests still pass with it in place, which shows it does not break them. Nothing measures the animation itself – XCUITest has no vocabulary for "this did not pop."
- Negative assertions (debounce non-firing) are bounded windows – they fail open under extreme load, and say so.
- Whether the pull-to-refresh gesture is reachable at all while `.searchToolBarBehavior(.minimize)` is on the same screen is *unmeasured*. Both claim the top overscroll – one to expand the search field, one to refresh – and which wins has not been checked here. The UI suite drives refreshes by re-submitting the query rather than by pulling, so nothing in it would notice; the source note at `SearchView`'s `.refreshable` says the same thing where a reader of the code will meet it.
- The Favorites screen is audited for accessibility at default text size only. Its rows are `RepoRowView`, so the reflow that rescues them at AX5 is literally the code the results-list AX5 test exercises – but "the same view type" is an argument, not a measurement, and the screen around it (an `EditButton`, a swipe-to-delete affordance) is neither.

Where to go next

Read `ARCHITECTURE.md` for the tour in prose, then the code in this order: `Support/LoadState.swift` → `ViewModels/SearchViewModel.swift` → `Services/LiveGitHubClient.swift` (the `@concurrent` comment especially) → `Views/Search/SearchView.swift` through its two private leaf views → `testExampleTests/TestSupport.swift` for the double roster. The doc comments are the second half of this guide – written, audited twice, and corrected to be read.